

## SIMATS ENGINEERING ASSESSMENT TOOL 2

**R.HARIKA**  
**192572140**

**COURSE NAME:** Database Management  
**Systems COURSE CODE:** CSA05

---

### COURSE OUTCOME COVERED

**CO2:** *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)  
Assessment Tool Weightage: 35%

---

Dr

| QUESTIONS |                                                                                                                                                       |               | Total Marks: 50 |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-----------------|
| Q.No      | Question                                                                                                                                              | Bloom's Level | Marks           |
| 1         | Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.           | BL3 – Apply   | 5               |
| 2         | A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month. | BL3 – Apply   | 5               |
| 3         | In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.                                         | BL3 – Apply   | 5               |
| 4         | For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.                            | BL3 – Apply   | 5               |
| 5         | Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.   | BL6 – Create  | 5               |
| 6         | In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.                       | BL3 – Apply   | 5               |

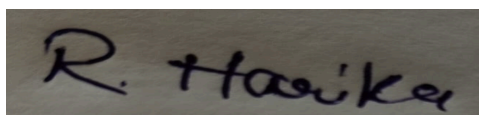
|    |                                                                                                                                       |              |   |
|----|---------------------------------------------------------------------------------------------------------------------------------------|--------------|---|
| 7  | A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000. | BL3 – Apply  | 5 |
| 8  | For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.    | BL3 – Apply  | 5 |
| 9  | Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.       | BL3 – Apply  | 5 |
| 10 | Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.           | BL6 – Create | 5 |

---

**Code of Conduct:**

*I R.HARIKA(192572140)\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**



**RUBRICS FOR CALCULATION:**

| Scenario based assessment    |               |                                                                           |                                                 |                                                   |                                                   |
|------------------------------|---------------|---------------------------------------------------------------------------|-------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| Criteria                     | Weightage (%) | Level 4 (Exemplary)                                                       | Level 3 (Proficient)                            | Level 2 (Developing)                              | Level 1 (Beginning)                               |
| Understanding of Scenario    | 20%           | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps        | Partial understanding; misses key aspects         | Misinterprets or fails to understand the scenario |
| Identification of Key Issues | 20%           | Accurately identifies all relevant issues and factors involved            | Identifies most key issues with minor omissions | Identifies limited issues; some irrelevant points | Unable to identify key issues                     |

|                         |     |                                             |                                    |                                          |                                   |
|-------------------------|-----|---------------------------------------------|------------------------------------|------------------------------------------|-----------------------------------|
| Application of Concepts | 25% | Applies appropriate concepts effectively to | Applies concepts with minor errors | Limited or partially correct application | Unable to apply relevant concepts |
|-------------------------|-----|---------------------------------------------|------------------------------------|------------------------------------------|-----------------------------------|

|                     |     |                                                              |                                                    |                                                 |                                               |
|---------------------|-----|--------------------------------------------------------------|----------------------------------------------------|-------------------------------------------------|-----------------------------------------------|
|                     |     | address the scenario                                         |                                                    |                                                 |                                               |
| Decision Making     | 20% | Makes well-reasoned, logical decisions with strong rationale | Makes reasonable decisions with some justification | Makes weak decisions with limited justification | No clear decisions made or rationale provided |
| Clarity of Response | 15% | Response is clear, structured, and logically presented       | Generally clear with minor issues                  | Somewhat unclear or poorly structured           | Disorganized and difficult to understand      |

### 1. School Management System

Identify relations and write SQL to list all students with their class teacher and grade.

#### Relations

##### STUDENT

**StudentID StudentName ClassID**

101      Rahul      C1

102      Priya      C2

103      Arjun      C1

##### CLASS

**ClassID ClassName TeacherID**

C1      X-A      T1

C2      X-B      T2

##### TEACHER

**TeacherID TeacherName**

T1      Mr. Kumar

T2      Mrs. Anita

##### GRADE

**StudentID Grade**

101      A

102      B+

103      A+

##### SQL Query

```
SELECT S.StudentName,
       C.ClassName,
       T.TeacherName,
       G.Grade
FROM STUDENT S
JOIN CLASS C
ON S.ClassID = C.ClassID
```

JOIN TEACHER T  
 ON C.TeacherID = T.TeacherID  
 JOIN GRADE G  
 ON S.StudentID = G.StudentID;

#### Output

| StudentName | ClassName | TeacherName | Grade |
|-------------|-----------|-------------|-------|
| Rahul       | X-A       | Mr. Kumar   | A     |
| Priya       | X-B       | Mrs. Anita  | B+    |
| Arjun       | X-A       | Mr. Kumar   | A+    |

## 2. Hospital Management System

Design the relational schema and write SQL to find doctors with more than 10 appointments this month.

#### Schema

##### DOCTOR

| DoctorID | DoctorName |
|----------|------------|
|----------|------------|

|    |           |
|----|-----------|
| D1 | Dr. Ravi  |
| D2 | Dr. Meena |

##### APPOINTMENT

| AppointmentID | DoctorID | AppointmentDate |
|---------------|----------|-----------------|
|---------------|----------|-----------------|

|     |     |            |
|-----|-----|------------|
| A1  | D1  | 2026-08-01 |
| A2  | D1  | 2026-08-02 |
| ... | ... | ...        |

#### SQL Query

```
SELECT D.DoctorID,
       D.DoctorName,
       COUNT(A.AppointmentID) AS TotalAppointments
FROM DOCTOR D
JOIN APPOINTMENT A
ON D.DoctorID = A.DoctorID
WHERE MONTH(A.AppointmentDate)=MONTH(CURRENT_DATE)
AND YEAR(A.AppointmentDate)=YEAR(CURRENT_DATE)
GROUP BY D.DoctorID,D.DoctorName
HAVING COUNT(A.AppointmentID)>10;
```

#### Sample Output

| DoctorID | DoctorName | TotalAppointments |
|----------|------------|-------------------|
| D1       | Dr. Ravi   | 15                |

## 3. E-Commerce System

Write SQL using sub-query to find products that have never been ordered.

##### PRODUCT

| ProductID | ProductName |
|-----------|-------------|
|-----------|-------------|

|    |        |
|----|--------|
| P1 | Laptop |
| P2 | Mobile |
| P3 | Tablet |

##### ORDERS

| OrderID | ProductID |
|---------|-----------|
|---------|-----------|

|    |    |
|----|----|
| O1 | P1 |
| O2 | P2 |

#### SQL Query

```
SELECT ProductID, ProductName
FROM PRODUCT
```

```
WHERE ProductID NOT IN
(
SELECT ProductID
FROM ORDERS
);
```

#### **Output**

**ProductID ProductName**

P3 Tablet

---

#### **4. University Database**

**Write Relational Algebra to find students enrolled in both DBMS and OS courses.**

##### **ENROLLMENT**

**StudentID Course**

101 DBMS

101 OS

102 DBMS

103 OS

##### **Relational Algebra**

**Students taking DBMS**

$\pi$  StudentID ( $\sigma$  Course='DBMS' (ENROLLMENT))

**Students taking OS**

$\pi$  StudentID ( $\sigma$  Course='OS' (ENROLLMENT))

**Students enrolled in both**

$\pi$  StudentID ( $\sigma$  Course='DBMS'(ENROLLMENT))

$\cap$

$\pi$  StudentID ( $\sigma$  Course='OS'(ENROLLMENT))

#### **Output**

StudentID

101

---

#### **5. Payroll System**

**Create a view showing employees whose salary is below department average and update their salary by 10%.**

##### **EMPLOYEE**

**EmpID Name DeptID Salary**

101 Rahul D1 40000

102 Arjun D1 50000

103 Priya D2 60000

104 Sneha D2 45000

#### **Create View**

```
CREATE VIEW LowSalaryEmployees AS
```

```
SELECT *
```

```
FROM EMPLOYEE E
```

```
WHERE Salary <
```

```
(
```

```
SELECT AVG(Salary)
```

```
FROM EMPLOYEE
```

```
WHERE DeptID=E.DeptID
```

```
);
```

#### **View Output**

**EmpID Name DeptID Salary**

101 Rahul D1 40000

104 Sneha D2 45000

#### **Update Query**

```

UPDATE EMPLOYEE
SET Salary=Salary*1.10
WHERE EmpID IN
(
SELECT EmpID
FROM LowSalaryEmployees
);

```

#### **Updated Output**

| <b>EmpID</b> | <b>Name</b> | <b>Salary</b> |
|--------------|-------------|---------------|
| 101          | Rahul       | 44000         |
| 104          | Sneha       | 49500         |

### **6. Banking System**

**Apply JOIN operations to retrieve customer account details, transaction history and branch information.**

#### **CUSTOMER**

| <b>CustomerID</b> | <b>CustomerName</b> | <b>AccountID</b> |
|-------------------|---------------------|------------------|
|-------------------|---------------------|------------------|

|     |       |      |
|-----|-------|------|
| 101 | Rahul | A101 |
| 102 | Priya | A102 |

#### **ACCOUNT**

| <b>AccountID</b> | <b>BranchID</b> | <b>Balance</b> |
|------------------|-----------------|----------------|
|------------------|-----------------|----------------|

|      |    |       |
|------|----|-------|
| A101 | B1 | 50000 |
| A102 | B2 | 75000 |

#### **TRANSACTION**

| <b>TransactionID</b> | <b>AccountID</b> | <b>Amount</b> | <b>Type</b> |
|----------------------|------------------|---------------|-------------|
|----------------------|------------------|---------------|-------------|

|      |      |       |            |
|------|------|-------|------------|
| T001 | A101 | 5000  | Deposit    |
| T002 | A101 | 2000  | Withdrawal |
| T003 | A102 | 10000 | Deposit    |

#### **BRANCH**

| <b>BranchID</b> | <b>BranchName</b> | <b>City</b> |
|-----------------|-------------------|-------------|
|-----------------|-------------------|-------------|

|    |            |         |
|----|------------|---------|
| B1 | Anna Nagar | Chennai |
| B2 | T. Nagar   | Chennai |

#### **SQL Query**

##### **SELECT**

```

C.CustomerName,
A.AccountID,
A.Balance,
T.TransactionID,
T.Amount,
T.Type,
B.BranchName,
B.City

```

```

FROM CUSTOMER C

```

```

INNER JOIN ACCOUNT A

```

```

ON C.AccountID=A.AccountID

```

```

INNER JOIN TRANSACTION T

```

```

ON A.AccountID=T.AccountID

```

```

INNER JOIN BRANCH B

```

```

ON A.BranchID=B.BranchID;

```

#### **Output**

| <b>Customer</b> | <b>Account</b> | <b>Balance</b> | <b>Transaction</b> | <b>Amount</b> | <b>Type</b> | <b>Branch</b> | <b>City</b> |
|-----------------|----------------|----------------|--------------------|---------------|-------------|---------------|-------------|
| Rahul           | A101           | 50000          | T001               | 5000          | Deposit     | Anna Nagar    | Chennai     |
| Rahul           | A101           | 50000          | T002               | 2000          | Withdrawal  | Anna Nagar    | Chennai     |
| Priya           | A102           | 75000          | T003               | 10000         | Deposit     | T. Nagar      | Chennai     |

## 7.Assumed Relations

CUSTOMER(Customer\_ID, Customer\_Name)

TRANSACTION(Transaction\_ID, Customer\_ID, Transaction\_Amount, Transaction\_Date)

### SQL Query

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    SUM(t.Transaction_Amount) AS Total_Transaction
FROM CUSTOMER c
JOIN TRANSACTION t
ON c.Customer_ID = t.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name;
```

**Note:** If your SQL software treats TRANSACTION as a reserved word, use TRANSACTIONS as the table name instead.

### Explanation

- JOIN combines the **CUSTOMER** and **TRANSACTION** tables using Customer\_ID.
- SUM() calculates the total transaction amount for each customer.
- GROUP BY groups the records by customer.
- The query displays each customer's name and their total transaction amount.

### Sample Output

| Customer_ID | Customer_Name | Total_Transaction |
|-------------|---------------|-------------------|
| C101        | Rahul         | 25,000            |
| C102        | Priya         | 18,500            |
| C103        | Arjun         | 32,000            |

7)A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000. (5 Marks)

### SALES Table

| SaleID | Category    | Amount |
|--------|-------------|--------|
| 1      | Electronics | 6000   |
| 2      | Electronics | 7000   |
| 3      | Clothing    | 5000   |
| 4      | Clothing    | 3000   |
| 5      | Grocery     | 12000  |
| 6      | Grocery     | 4000   |

### SQL Query

```
SELECT Category,
    SUM(Amount) AS TotalSales
FROM SALES
GROUP BY Category
HAVING SUM(Amount) > 10000;
```

### Output

| Category    | TotalSales (Rs.) |
|-------------|------------------|
| Electronics | 13000            |
| Grocery     | 16000            |

### Explanation

- GROUP BY Category groups the records by product category.
- SUM(Amount) calculates the total sales for each category.

- HAVING SUM(Amount) > 10000 displays only those categories whose total sales exceed Rs. 10,000.

### 8)Library Management System

Construct Tuple Relational Calculus (TRC) expressions to find members who borrowed books from all genres.

#### MEMBER

**MemberID MemberName**

|    |       |
|----|-------|
| M1 | Rahul |
| M2 | Priya |
| M3 | Arjun |

#### BOOK

**BookID Title Genre**

|    |         |            |
|----|---------|------------|
| B1 | DBMS    | Technology |
| B2 | Hamlet  | Literature |
| B3 | Physics | Science    |

#### BORROW

**MemberID BookID**

|    |    |
|----|----|
| M1 | B1 |
| M1 | B2 |
| M1 | B3 |
| M2 | B1 |
| M2 | B2 |
| M3 | B1 |

Tuple Relational Calculus (TRC)

```
{ M.MemberName |
  MEMBER(M) ∧
  ∀G ( ∃B (BOOK(B) ∧ B.Genre = G)
    →
    ∃BR (BORROW(BR) ∧
      BR.MemberID = M.MemberID ∧
      ∃BK (BOOK(BK) ∧
        BK.BookID = BR.BookID ∧
        BK.Genre = G)))
}
```

#### Output

MemberName  
Rahul

### 9. Inventory Management System:

Define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.

#### CATEGORY Table

```
CREATE TABLE CATEGORY (
  CategoryID INT PRIMARY KEY,
  CategoryName VARCHAR(50)
);
```

#### PRODUCT Table

```
CREATE TABLE PRODUCT (
  ProductID INT PRIMARY KEY,
  ProductName VARCHAR(50),
  Price DECIMAL(10,2) CHECK (Price > 0),
```



```

    Stock INT CHECK (Stock >= 0),
    CategoryID INT,
    FOREIGN KEY (CategoryID)
    REFERENCES CATEGORY(CategoryID)
);

```

#### Insert Valid Data

```

INSERT INTO CATEGORY VALUES
(101,'Electronics'),
(102,'Furniture');

```

```

INSERT INTO PRODUCT VALUES
(1,'Laptop',55000,10,101),
(2,'Chair',3000,20,102);

```

#### Output

2 rows inserted into CATEGORY.

2 rows inserted into PRODUCT.

#### CHECK Constraint Example

```

INSERT INTO PRODUCT
VALUES (3,'Mobile',-15000,5,101);

```

#### Output

ERROR:

CHECK constraint violated.

Price must be greater than 0.

#### FOREIGN KEY Constraint Example

```

INSERT INTO PRODUCT
VALUES (4,'Table',5000,10,105);

```

#### Output

ERROR:

FOREIGN KEY constraint fails.

CategoryID 105 does not exist.

#### PRODUCT Table

| ProductID | ProductName | Price | Stock | CategoryID |
|-----------|-------------|-------|-------|------------|
| 1         | Laptop      | 55000 | 10    | 101        |
| 2         | Chair       | 3000  | 20    | 102        |

#### Explanation

- **CHECK** ensures valid values (Price > 0, Stock >= 0).
- **FOREIGN KEY** ensures that every product belongs to an existing category.

### 10) HR Management System

Design a relational schema and create materialized views for monthly attendance and performance summaries.

#### EMPLOYEE

| EmployeeID | EmployeeName | Department | Designation |
|------------|--------------|------------|-------------|
| 101        | Rahul        | HR         | Manager     |
| 102        | Priya        | IT         | Developer   |

#### ATTENDANCE

| AttendanceID | EmployeeID | Month   | DaysPresent |
|--------------|------------|---------|-------------|
| A1           | 101        | January | 26          |
| A2           | 102        | January | 24          |

## PERFORMANCE

| PerformanceID | EmployeeID | Month | Rating |
|---------------|------------|-------|--------|
|---------------|------------|-------|--------|

|    |     |         |     |
|----|-----|---------|-----|
| P1 | 101 | January | 4.8 |
|----|-----|---------|-----|

|    |     |         |     |
|----|-----|---------|-----|
| P2 | 102 | January | 4.5 |
|----|-----|---------|-----|

## SQL DDL

```
CREATE TABLE EMPLOYEE (  
    EmployeeID INT PRIMARY KEY,  
    EmployeeName VARCHAR(50),  
    Department VARCHAR(30),  
    Designation VARCHAR(30)  
);
```

```
CREATE TABLE ATTENDANCE (  
    AttendanceID VARCHAR(10) PRIMARY KEY,  
    EmployeeID INT,  
    Month VARCHAR(20),  
    DaysPresent INT,  
    FOREIGN KEY(EmployeeID)  
    REFERENCES EMPLOYEE(EmployeeID)  
);
```

```
CREATE TABLE PERFORMANCE (  
    PerformanceID VARCHAR(10) PRIMARY KEY,  
    EmployeeID INT,  
    Month VARCHAR(20),  
    Rating DECIMAL(3,1),  
    FOREIGN KEY(EmployeeID)  
    REFERENCES EMPLOYEE(EmployeeID)  
);
```

## Materialized View – Attendance

```
CREATE MATERIALIZED VIEW Monthly_Attendance AS  
SELECT EmployeeID,  
    Month,  
    SUM(DaysPresent) AS TotalDaysPresent  
FROM ATTENDANCE  
GROUP BY EmployeeID, Month;
```

## Materialized View – Performance

```
CREATE MATERIALIZED VIEW Monthly_Performance AS  
SELECT EmployeeID,  
    Month,  
    AVG(Rating) AS AverageRating  
FROM PERFORMANCE  
GROUP BY EmployeeID, Month;
```

## Output – Monthly\_Attendance

| EmployeeID | Month | TotalDaysPresent |
|------------|-------|------------------|
|------------|-------|------------------|

|     |         |    |
|-----|---------|----|
| 101 | January | 26 |
|-----|---------|----|

|     |         |    |
|-----|---------|----|
| 102 | January | 24 |
|-----|---------|----|

## Output – Monthly\_Performance

| EmployeeID | Month | AverageRating |
|------------|-------|---------------|
|------------|-------|---------------|

|     |         |     |
|-----|---------|-----|
| 101 | January | 4.8 |
|-----|---------|-----|

|     |         |     |
|-----|---------|-----|
| 102 | January | 4.5 |
|-----|---------|-----|